Comparing ARIMA and LSTM for Short-Term Forecasting of Daily Average Uber Trip Duration in NYC

Farooq Mahmud
IDC 6940 - Capstone
University of West Florida

March 2026

Agenda

1. **Background & Motivation** - Why this problem matters
2. **Data** - NYC TLC High-Volume FHV data
3. **Mathematical Framework** - ARIMA and LSTM models
4. **Experimental Design** - Train / validation / test split & metrics
5. **Code & Implementation** - Sources, challenges, accomplishments
6. **Results** - Forecasts and metric comparison
7. **Conclusion & Future Work**

The Research Question

> **How do ARIMA and LSTM forecasting methods compare for 14-day forecasting of daily average Uber trip duration in New York City when evaluated on sMAPE and MASE?**

Why it matters

- Operational planning: driver scheduling, surge pricing, fleet management
- Classical vs. neural: is extra complexity worth it for short horizons?
- Practical benchmark on a real-world urban-mobility time series

Scope

Item	Detail
Target variable	Daily average Uber trip duration (minutes)
Data	366 daily observations, full year 2024
Forecast horizon	14 days
Evaluation	sMAPE and MASE on the same test window

Background & Motivation

Classical: ARIMA

- Decades of proven performance on univariate time series
- Interpretable parameters: autoregressive, differencing, moving-average orders
- Model selection via AIC and BIC; stationarity tested with ADF

Neural: LSTM

- Hochreiter & Schmidhuber (1997) - solves the vanishing gradient problem
- Memory cells with forget, input, and output gates capture long-range dependencies
- Increasingly popular for financial, weather, and transportation forecasting

The gap this project fills

- Direct, reproducible comparison on the **same** real-world urban-mobility series
- Same training data, same test window, same evaluation metrics
- Small dataset (366 obs): does deep learning help or hurt?

> Prior comparative studies (Prabhat et al., 2024) show mixed results - this project provides a concrete, controlled benchmark.

Data: NYC TLC High-Volume FHV

Source

NYC Taxi & Limousine Commission - High-Volume FHV Trip Records (2024)

data.nyc.gov

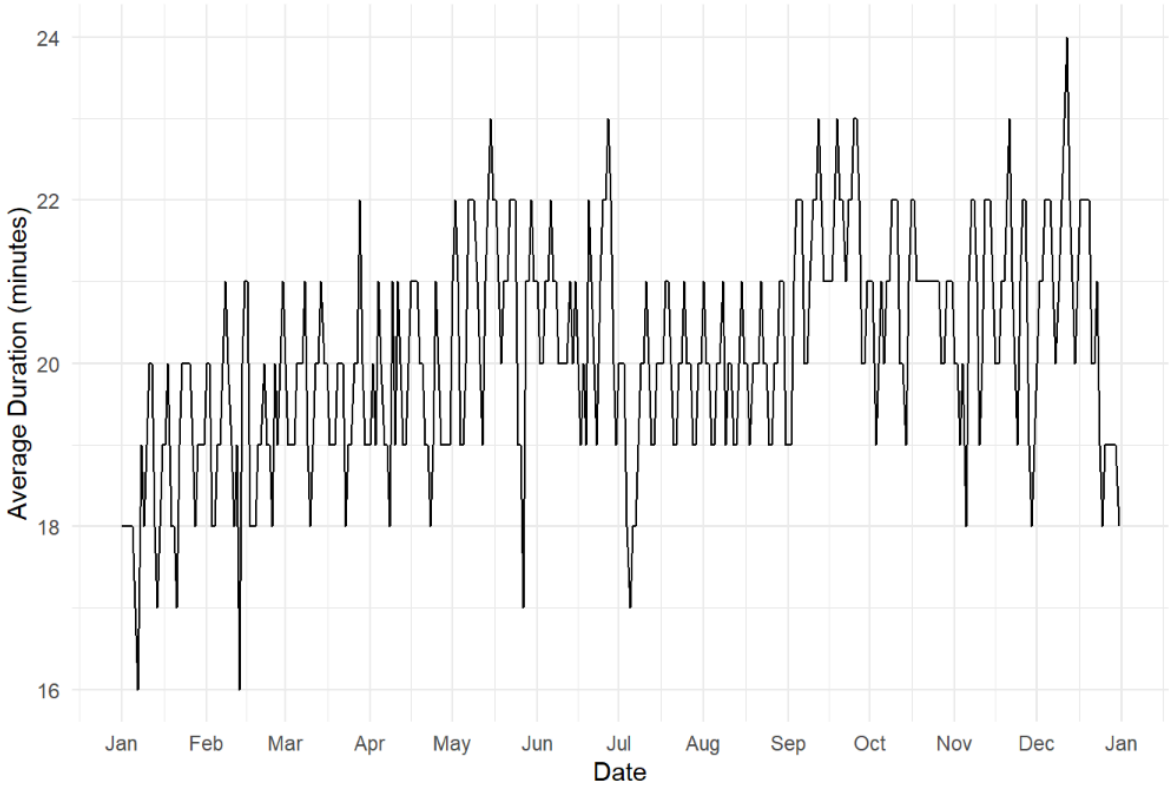
Scale of raw data

Stage	Row count
All HVFHV records	~200 million
After Uber filter (`HV0003`)	~179 million
After outlier removal	~159 million
Final time series	**366 rows (1 per day)**

Preprocessing pipeline (PySpark on Google Colab)

1. Filter to Uber (`hvfhs_license_num == "HV0003"`)
2. Derive `trip_duration_min` and `avg_speed_mph`
3. Range filters: duration 1-120 min, distance 0.1-100 mi, speed 1-80 mph
4. IQR-based outlier removal using `approxQuantile`
5. Group by date -> mean duration per day -> write to CSV

The Time Series



- **366 daily observations**, January 1 - December 31, 2024
- Visible **weekly seasonality**: weekday trips longer than weekend trips
- Holiday dips visible (New Year's, July 4th, Thanksgiving, Christmas)
- Series is relatively stationary - favorable for ARIMA

Mathematical Framework: ARIMA

ARIMA(p, d, q)

$$\phi(B)(1-B)^d y_t = \theta(B)\varepsilon_t$$

Symbol	Meaning
B	Backshift operator: $By_t = y_{t-1}$
d	Differencing order (achieves stationarity)
$\phi(B)$	AR polynomial of order p
$\theta(B)$	MA polynomial of order q
ε_t	White noise

Model selected: ARIMA(1, 1, 4)

- **d = 1**: One round of differencing removes trend -> stationarity
- **p = 1**: Prediction depends on the most recent observation
- **q = 4**: Corrections based on forecast errors from the last 4 days
- Selected automatically by `auto.arima` using **AIC and BIC minimization**

Mathematical Framework: LSTM

Training loss (MSE)

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

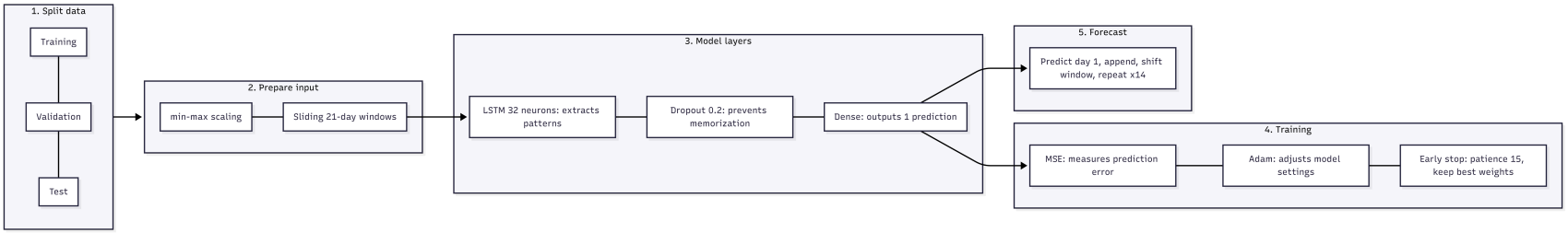
Architecture

Component	Setting
Input shape	(21 time steps × 1 feature)
LSTM layer	32 neurons
Dropout	rate = 0.2
Dense output	1 neurons
Optimizer	Adam
Loss	MSE
Early stopping	patience = 15, restore best weights

Recursive 14-day forecast

Feed last 21-day window -> predict day 1 -> append -> roll window -> repeat 14×

LSTM Architecture



Experimental Design

Chronological data split (366 days)

```
|<--- Training: 322 days --->|<-- Validation: 30 days -->|<-- Test: 14 days -->|
 Jan 1, 2024                                              Dec 18        Dec 31
```

No future data leaks into training - strict temporal ordering enforced.

Evaluation metrics

sMAPE (symmetric mean absolute percentage error):

$$\text{sMAPE} = \frac{100\%}{n} \sum_{t=1}^{n} \frac{|y_t - \hat{y}_t|}{(|y_t| + |\hat{y}_t|)/2}$$

MASE (mean absolute scaled error - scaled by naive 1-step MAE on training data):

$$\text{MASE} = \frac{\text{MAE}_{\text{test}}}{\text{MAE}_{\text{naive, train}}}$$

- MASE < 1 -> beats naive (uses last known value) baseline | MASE > 1 -> worse than naive

Code & Implementation

Origins

Component	Source
PySpark preprocessing	written from scratch using PySpark docs & NYC TLC data dictionary
ARIMA pipeline (R)	uses standard `forecast::auto.arima` library functions
LSTM pipeline (R/Keras)	adapts documented Keras time-series patterns to R
ggplot2 visualization	standard ggplot2 idioms

Closest public references

- **rwanjohi/Time-series-forecasting-using-LSTM-in-R** - similar concept, different API (sequential vs. functional) and preprocessing approach
- **keras.io timeseries forecasting tutorial** - sliding-window + LSTM pattern (Python); adapted to R

Key accomplishment

Full end-to-end reproducible pipeline: 200M-row PySpark ETL -> R ARIMA -> R LSTM -> metric comparison

Implementation Challenges

Challenge 1: LSTM reproducibility

- LSTM training is **stochastic**: weight initialization, random validation split, dropout
- Setting `tensorflow::set_random_seed()` and `set.seed()` did **not** produce consistent results in our R/Windows/Keras environment
- sMAPE and MASE values shifted noticeably run-to-run

Solution: Run LSTM 5 independent times -> report **mean ± standard deviation**

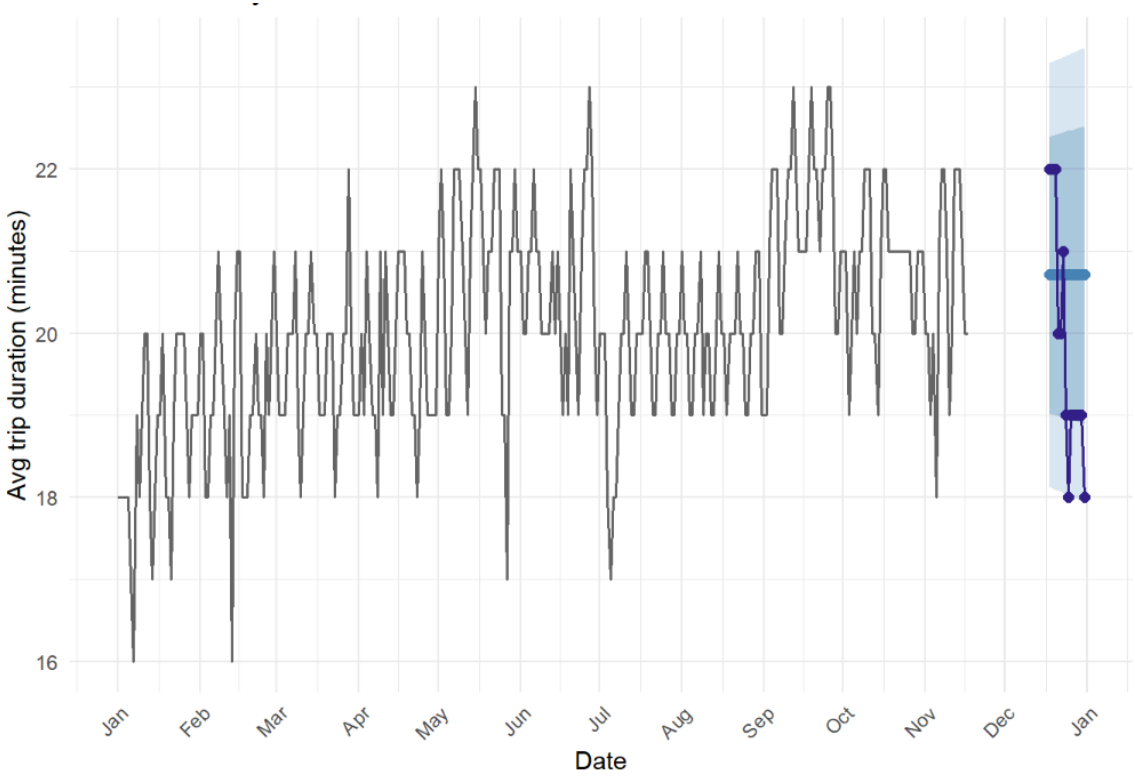
Challenge 2: PySpark on Google Colab

- Google Colab chosen to avoid local PySpark setup - ready-to-use environment with `pip install pyspark`
- Full pipeline ran in **~14 minutes** on 200M rows; IQR (`approxQuantile`) alone took **~6 min (43%)**
- `approxQuantile` is approximate by design - fast enough at this scale, acceptable trade-off

Key accomplishments

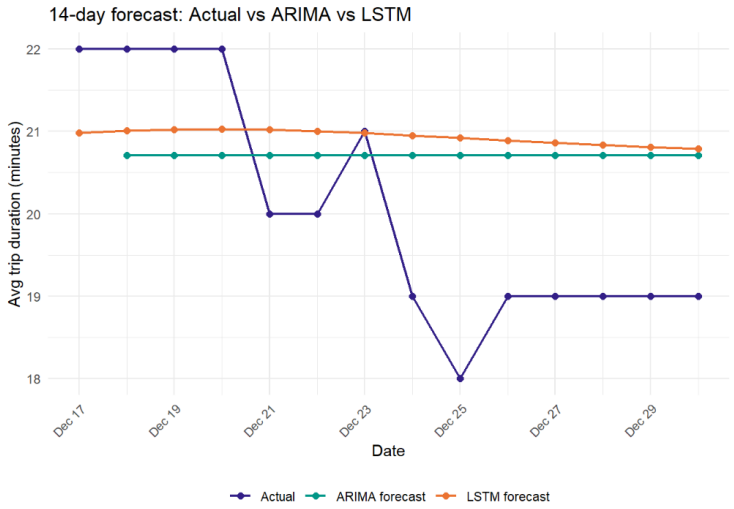
- End-to-end reproducible Quarto pipelines for both models
- Honest uncertainty quantification for LSTM metrics
- Fair evaluation: identical test window for both models

Results: ARIMA 14-Day Forecast



- **Flat forecast**: ARIMA(1,1,4) converges toward recent average - expected behavior
- **Intervals**: Most actuals fall within 95% band - reasonable calibration
- **Christmas dip** (Dec 25) falls near lower 80% bound - holiday effect not captured

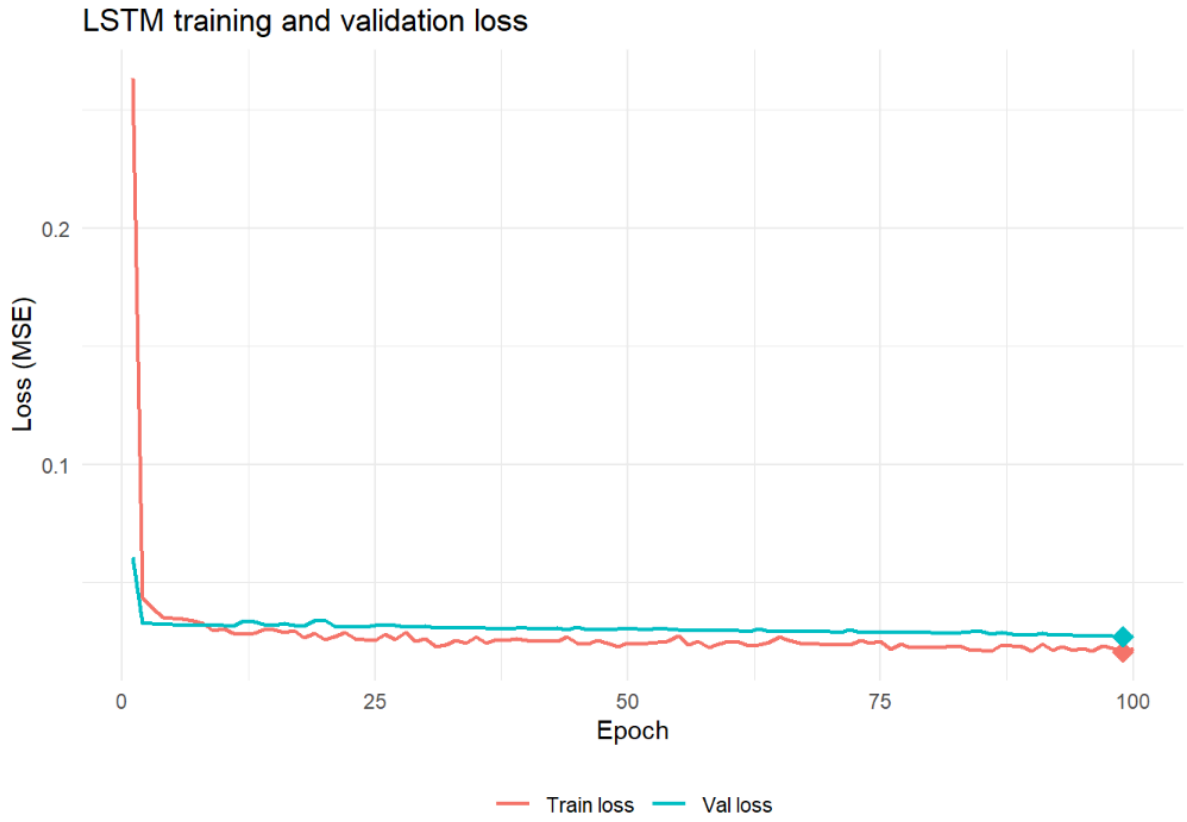
Results: ARIMA vs LSTM



Model	sMAPE (%)	MASE
ARIMA	**7.58**	**1.93**
LSTM	8.05 ± 0.34	2.09 ± 0.09

- ARIMA **slightly outperforms** LSTM on both metrics
- LSTM values are mean ± SD of **5 independent runs**
- Both MASE > 1: neither beats the naive forecast on this test window
- Neither model captures the **Christmas Day dip** (Dec 25)

Results: LSTM Training Convergence



- Both curves decrease quickly and **stabilize** - no overfitting
- Markers (◆) show epochs with minimum training and validation loss
- Early stopping fired before 100 epochs - sufficient number of epochs
- Stable convergence -> poor forecast accuracy is a **data limitation**, not a training problem

Discussion & Interpretation

Why ARIMA performed similarly to (or better than) LSTM

Factor	Impact
Small dataset (366 obs, ~280 training)	Limits LSTM learning capacity
Univariate - no external features	Both models blind to holidays/weather
Short horizon (14 days)	ARIMA's flat forecast surprisingly competitive
Stable dynamics	Simple AR + MA structure sufficient

What neither model could do

- Capture the **Christmas Day anomaly** (Dec 25 sharp dip)
- Adapt to holiday calendar effects without additional variables

Key insight

> The assumption that "deep learning is always better" does **not** hold in small-data, short-horizon settings. ARIMA's simplicity is a strength when data are limited.

Conclusion

Research question answered

On the same 14-day holdout with the same metrics, **ARIMA slightly outperformed LSTM**:

	sMAPE	MASE
ARIMA	7.58%	1.93
LSTM (mean ± SD, 5 runs)	8.05% ± 0.34	2.09 ± 0.09

Contributions

- Fully reproducible end-to-end pipeline (PySpark -> R ARIMA -> R LSTM -> metrics)
- Honest LSTM uncertainty quantification (mean ± SD over 5 runs)
- Fair comparison: identical data, splits, test window, and metrics

Takeaway

> Classical ARIMA is competitive with-or better than-LSTM for **short-horizon, univariate, small-dataset** forecasting. Complexity is not always rewarded.

Future Work

1. More data

- Extend to 3-4 years of NYC TLC data -> more LSTM training sequences
- Reasses ARIMA vs LSTM

2. External variables

- Add **weather** (temperature, precipitation), **holiday indicators**, **day-of-week** features
- Especially beneficial for LSTM (handles multivariate inputs naturally)

3. Reproducibility

- Explore deterministic training options in TF

Thank You

Code, data, and full outputs available at:

github.com/farooq-teqniqly/uwf-idc6940-capstone-files

References

- Hyndman & Khandakar (2008) - `forecast` R package, auto.arima
- Hochreiter & Schmidhuber (1997) - LSTM
- Prabhat et al. (2024) - ARIMA vs ML comparative study
- NYC TLC (2024) - High-Volume FHV Trip Records